

0.10. Tout l'alphabet, avec poserS

poserS passe à la ligne tout seul : une seule boucle pour les 26 lettres.

1. Ce qu'il faut comprendre

Ce qui est nouveau ici : `poserS`. Le reste (`for`, `sizeof`, `ALPHABET[i]`) vient des chapitres précédents : c'est le programme du 0.9, en plus simple.

`sizeof(ALPHABET)` veut dire « la taille de ALPHABET » : le nombre de cases du tableau, **26**. On ne l'écrit plus à la main : si le tableau changeait de taille, la boucle suivrait toute seule. `i < sizeof(ALPHABET)` s'arrête donc après `i = 25`, la dernière lettre.

`poserS(colonne, ligne, tuile)` fait partie de la console, comme `ALPHABET` : on s'en sert sans rien déclarer. C'est `poser()`, mais qui **passe à la ligne tout seul** : une colonne trop grande (20 ou plus) repart à gauche, une ligne plus bas.

On lui donne donc simplement `i` comme colonne, sur la ligne 0. Tour par tour : **`i = 0`** → A en (0, 0) · **`i = 19`** → T en (19, 0), la ligne est pleine · **`i = 20`** → la colonne 20 n'existe pas : `poserS` pose U en (**0**, 1) · **`i = 25`** → Z en (5, 1).

Comment `poserS` trouve la place, pour les curieux : la ligne est le nombre de rangées de 20 déjà pleines ($20 / 20 = 1$), et la colonne ce qui reste une fois ces rangées retirées ($25 - 20 = 5$). En C++ : `i / 20` et `i % 20`. `poserS` fait ce calcul à notre place.

Le calcul du 0.9 a disparu du programme : `poserS` le fait à notre place.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1360 octets** de cartouche.

1. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

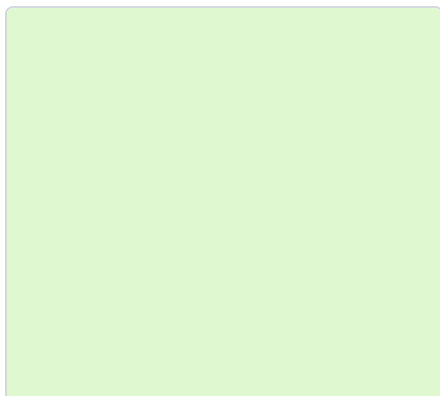
```
int main() {
    // for : répète le bloc entre { } ; i part de 0 et augmente de 1 à chaque tour.
    // sizeof(ALPHABET) = 26, la taille du tableau : 26 tours, i va de 0 à 25.
    for (uint8_t i = 0; i < sizeof(ALPHABET); i++) {

        // poserS(colonne, ligne, tuile) : comme poser(), mais passe à la ligne tout seul.
        // i          = la colonne : de 0 à 19 elle tient sur la ligne 0 ;
        //            à partir de 20, poserS repart à gauche une ligne plus bas
        //            (i = 20 → U en (0, 1), i = 25 → Z en (5, 1))
        // 0          = la ligne de départ
        // ALPHABET[i] = la lettre n° i : A pour i = 0, Z pour i = 25
        poserS(i, 0, ALPHABET[i]);
    }

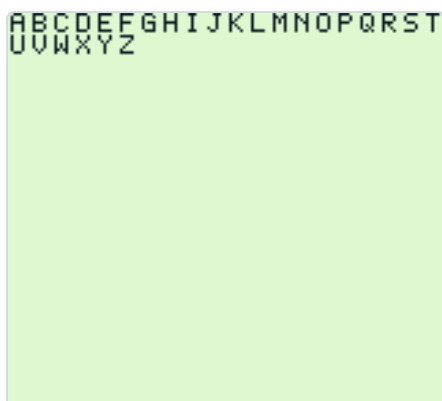
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

A à T sur la ligne 0, U à Z en dessous, comme au 0.8.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ la ligne 0 : A à T

✓ la ligne 1 : U à Z

Ensuite : **0.10.1. Tout l'alphabet, avec poserS — de base, ailleurs** — niveau 0 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++